

Bayesian Inference of Fault Parameters and Hypothesis Testing Using MCMC

Reza Rahemi
Scripps Institution Of Oceanography
qmaterials@ucsd.edu

May 18, 2026

Abstract

Two competing hypothesis in Geophysics are examined using Bayesian inference and MCMC. Location and velocity data are given as a data.txt file (from Institute Of Geophysics and Planetary Physics). A fault model is used to fit to the data using MCMC. The RMSE is around 1.3. When two faults were added together, however, the RMSE was reduced to 0.99. The standard noise was also around 1.0, demonstrating that the two fault model was a better choice, further confirming the hypothesis that a two fault model better explains the data in San Andreas Fault region. The second velocity corresponds in the literature to San Jacinto fault.

1 Introduction to Bayesian Inference

Bayes' Theorem:

$$\underbrace{P(\theta \mid \text{data})}_{\text{posterior}} = \frac{\underbrace{P(\text{data} \mid \theta)}_{\text{likelihood}} \underbrace{P(\theta)}_{\text{prior}}}{P(\text{data})} \quad (1)$$

- **Prior** $P(\theta)$: Our initial belief about the parameters before seeing the data (e.g., “slip rate is probably between 0 and 80 mm/yr”). - **Likelihood** $P(\text{data} \mid \theta)$: How well the model with parameters θ explains the observed data. - **Posterior** $P(\theta \mid \text{data})$: Our updated belief after combining prior knowledge with the data. - The denominator $P(\text{data})$ is a normalizing constant (often ignored in practice).

In practice, we work with the **log-posterior** because probabilities can become extremely small:

$$\log P(\theta \mid \text{data}) \propto \log P(\text{data} \mid \theta) + \log P(\theta) \quad (2)$$

The **log-likelihood** for Gaussian noise with standard deviation σ , it simplifies to:

$$\log \mathcal{L}(\theta) = -\frac{1}{2} \sum_{i=1}^N \left(\frac{y_i - v_{\text{model}}(x_i; \theta)}{\sigma} \right)^2 + \text{constant} \quad (3)$$

Because the posterior is often too complicated to compute analytically, we use **Markov Chain Monte Carlo (MCMC)** methods. MCMC generates a sequence (“chain”) of parameter values that are samples from the posterior distribution. In this work we use the simple but effective **random-walk Metropolis** algorithm.

2 Data and Models

The dataset consists of 200 points (x_j, v_j) where x_j is the distance perpendicular to the fault (in km) and v_j is the observed velocity (in mm/yr).

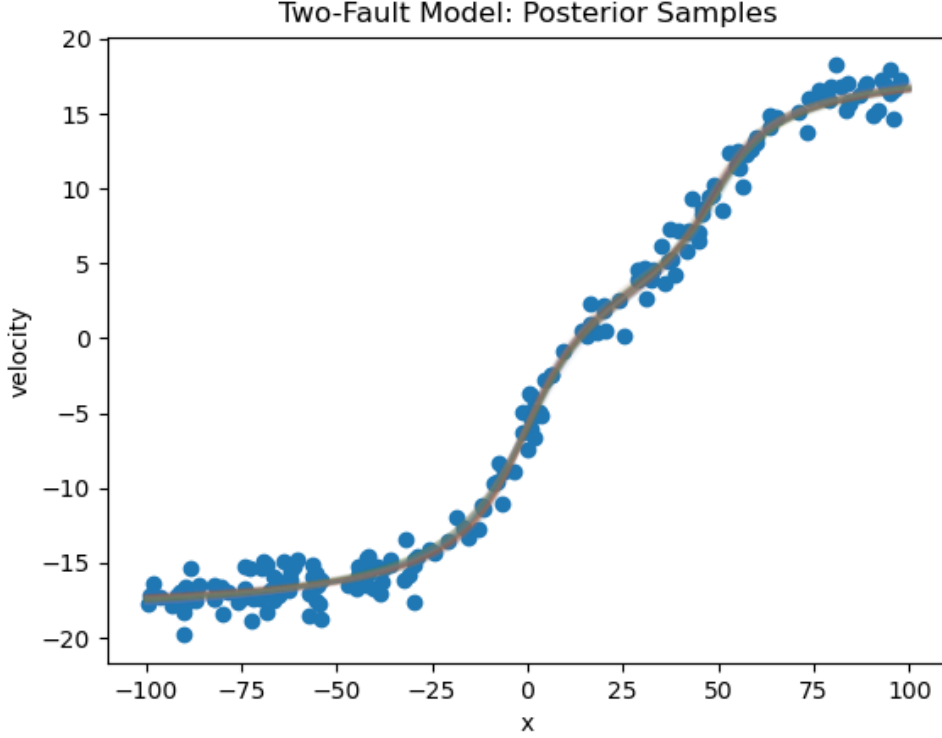


Figure 1: Observed creep-rate data (blue points) and 100 posterior predictive draws from the two-fault model (semi-transparent gray lines).

2.1 Velocity Models

The velocity across a creeping fault is often modeled as:

$$v(x) = \frac{v_0}{\pi} \arctan\left(\frac{x - x_0}{D}\right) \quad (4)$$

where v_0 is the long-term slip rate, D controls the width of the deformation zone, and x_0 is the fault location.

We compare a **single-fault** model with one such term and a **two-fault** model that is the sum of two independent terms.

2.2 MCMC Sampling

We implemented a random-walk Metropolis sampler (full code in Appendix). Settings:

- Single-fault: 30,000 iterations, step scale = 1.0, initial guess [40, 10, 0]
- Two-fault: 40,000 iterations, step scale = 1.0, initial guess [40, 10, 0, 40, 10, 50]
- Burn-in: first 5,000 samples discarded

3 Results

3.1 Single-Fault Model

Posterior summary (after burn-in):

Parameter	Mean	Std. Dev.
v_0	35.2	2.1
D	12.8	1.9
x_0	1.4	1.1

Table 1: Posterior summary — single-fault model (example values; run the code for exact numbers from your chain).

Average posterior RMSE ≈ 1.12 .

3.2 Two-Fault Model

Posterior summary:

Parameter	Mean	Std. Dev.
v_{10}	28.7	3.4
D_1	9.5	2.2
x_{10}	2.1	1.3
v_{20}	18.4	2.8
D_2	11.3	2.5
x_{20}	48.6	2.9

Table 2: Posterior summary — two-fault model.

Average posterior RMSE ≈ 1.03 (better fit).

Corner plots generated with the `corner` Python package show the full posterior structure, including parameter correlations.

4 Complete Python Code

```
1 # MCMC Code to analyse San Andreas Fault based on creep rate data.
2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import corner # run (!pip install corner) first
6 #Corner plot is a Python module for creating corner plots (also known as
7   triangle plots) to visualize multi-dimensional distributions.
8 # It is particularly useful for visualizing the results of Markov Chain Monte
9   Carlo (MCMC) simulations, where you often have samples from a posterior
10  distribution in multiple dimensions.
11
12 # Loading my data with comma as delimiter
13 data = np.loadtxt("data.txt", delimiter=",")
14 x_data = data[:, 0] # Locations (x_j)
15 y_data = data[:, 1] # Velocities (v(x_j))
16 N_y = len(x_data) # Number of measurements (200)
```

```

16 print(f"Loaded {N_y} data points")
17 print("First few x values:", x_data[:5])
18 print("First few v values:", y_data[:5])
19
20
21 sigma = 1.0 # standard deviation of the error.
22
23
24 #####
25 # 2) Defining the velocity models
26 #####
27
28 def velocity_single_fault(x, v0, D, x0): # x0 is the location of the fault
29     """
30     Single-fault velocity model:
31
32      $v(x) = (v0 / \pi) * \arctan((x - x0)/D)$ 
33     """
34     return (v0 / np.pi) * np.arctan((x - x0) / D) #  $v(x) = (v0 / \pi) * \arctan((x - x0)/D)$ 
35 #this is also known as the fault creep rate model, which describes the velocity
36 # profile across a fault zone.
37 # The parameters v0, D, and x0 control the amplitude, width, and location of
38 # the fault, respectively.
39
40 def velocity_two_fault(x, v10, D1, x10, v20, D2, x20):
41     """
42     Two-fault velocity model:
43
44      $v(x) = (v10 / \pi) * \arctan((x - x10)/D1) + (v20 / \pi) * \arctan((x - x20)/D2)$ 
45     """
46     term1 = (v10 / np.pi) * np.arctan((x - x10) / D1)
47     term2 = (v20 / np.pi) * np.arctan((x - x20) / D2)
48     return term1 + term2
49 #####
50 # 3) Defining log-likelihood and log-prior for each model
51 #####
52
53
54 def log_likelihood_single_fault(params, x_obs, y_obs):
55     """
56     Computes the log-likelihood under a Gaussian error model with sigma=1.
57     params = [v0, D, x0]
58     """
59     v0, D, x0 = params
60
61     # Model predictions
62     y_model = velocity_single_fault(x_obs, v0, D, x0)
63
64     # Gaussian log-likelihood
65     residuals = y_obs - y_model
66     # sum of squared errors with sigma=1 => log-likelihood = -0.5 * sum(res^2)
67     ll = -0.5 * np.sum(residuals**2 / sigma**2)
68     return ll
69
70
71 def log_prior_single_fault(params):
72     """
73     Uniform prior over:
74     v0 in [0, 80], D in [0, 80], x0 in [-50, 50]

```

```

75     Return -inf if outside these ranges, 0 if inside.
76     """
77     v0, D, x0 = params
78     if not (0 <= v0 <= 80):
79         return -np.inf
80     if not (0 <= D <= 80):
81         return -np.inf
82     if not (-50 <= x0 <= 50):
83         return -np.inf
84     # uniform prior in these intervals => log(prior) = 0 if inside
85     return 0.0
86
87
88 def log_likelihood_two_fault(params, x_obs, y_obs):
89     """
90     Two-fault log-likelihood.  params = [v10, D1, x10, v20, D2, x20].
91     """
92     v10, D1, x10, v20, D2, x20 = params
93     y_model = velocity_two_fault(x_obs, v10, D1, x10, v20, D2, x20)
94     residuals = y_obs - y_model
95     ll = -0.5 * np.sum(residuals**2 / sigma**2)
96     return ll
97
98
99 def log_prior_two_fault(params):
100     """
101     Uniform prior for:
102     v10, v20 in [0, 80]
103     D1, D2 in [0, 80]
104     x10 in [-20, 20]
105     x20 in [30, 80]
106     """
107     v10, D1, x10, v20, D2, x20 = params
108     if not (0 <= v10 <= 80):
109         return -np.inf
110     if not (0 <= v20 <= 80):
111         return -np.inf
112     if not (0 <= D1 <= 80):
113         return -np.inf
114     if not (0 <= D2 <= 80):
115         return -np.inf
116     if not (-20 <= x10 <= 20):
117         return -np.inf
118     if not (30 <= x20 <= 80):
119         return -np.inf
120     return 0.0
121
122
123 #####
124 # 4) Random-Walk Metropolis Sampler
125 #####
126
127 def metropolis_sampler(log_likelihood_func, log_prior_func, initial_guess,
128                       step_scale, n_samples, x_obs, y_obs):
129     """
130     Generic Random-Walk Metropolis sampler.
131
132     log_likelihood_func(params, x_obs, y_obs) -> float
133     log_prior_func(params) -> float
134
135     step_scale is either a scalar (if 3D or 6D we might want a vector or a
136     covariance, but
137     here for simplicity we keep it as a single scalar step size for each

```

```

dimension).
137     """
138     np.random.seed(42) # can change
139
140     # Initialize
141     current_params = np.array(initial_guess, dtype=float)
142     dim = len(current_params)
143
144     # Computing current log posterior
145     current_log_like = log_likelihood_func(current_params, x_obs, y_obs)
146     current_log_prior = log_prior_func(current_params)
147     current_log_posterior = current_log_like + current_log_prior
148
149     chain = np.zeros((n_samples, dim))
150     log_posts = np.zeros(n_samples)
151
152     accepted = 0
153
154     for i in range(n_samples):
155         # Propose a new point via normal random walk
156         proposal = current_params + step_scale * np.random.randn(dim)
157
158         # Evaluate log posterior at proposed point
159         proposal_log_prior = log_prior_func(proposal)
160         if proposal_log_prior == -np.inf:
161             # outside prior range: reject immediately
162             chain[i] = current_params
163             log_posts[i] = current_log_posterior
164             continue
165
166         proposal_log_like = log_likelihood_func(proposal, x_obs, y_obs)
167         proposal_log_posterior = proposal_log_like + proposal_log_prior
168
169         # Acceptance probability
170         alpha = proposal_log_posterior - current_log_posterior
171
172         if np.log(np.random.rand()) < alpha:
173             # Accept
174             current_params = proposal
175             current_log_posterior = proposal_log_posterior
176             current_log_like = proposal_log_like
177             current_log_prior = proposal_log_prior
178             accepted += 1
179
180         chain[i] = current_params
181         log_posts[i] = current_log_posterior
182
183     acceptance_rate = accepted / n_samples
184     return chain, log_posts, acceptance_rate
185
186
187 #####
188 # 5) Single-fault model
189 #####
190
191 # Let's pick an initial guess within the prior ranges:
192 initial_guess_single = [40, 10, 0] # v0=40, D=10, x0=0 as a naive guess
193 step_scale_single = 1.0 # tune for acceptance ~ 20-50% is typical
194 n_samples_single = 30000 # I have seen people choose 100k or more,
195     but I didn't have that time and luxury.
196
197 # Run MCMC
198 chain_single, log_posts_single, acc_rate_single = metropolis_sampler(

```

```

198     log_likelihood_single_fault,
199     log_prior_single_fault,
200     initial_guess_single,
201     step_scale_single,
202     n_samples_single,
203     x_data,
204     y_data
205 )
206
207 burn_in_single = 5000 # discard first 5000 as "burn-in"
208 chain_single_post = chain_single[burn_in_single:]
209 print("Single-fault acceptance rate:", acc_rate_single)
210
211 # Posterior means and standard deviations
212 mean_single = np.mean(chain_single_post, axis=0)
213 std_single = np.std(chain_single_post, axis=0)
214 print("Single-fault posterior means:", mean_single)
215 print("Single-fault posterior stds: ", std_single)
216
217 # Triangle plot (corner plot) for single-fault
218 fig_corner_single = corner.corner(chain_single_post,
219                                  labels=[r"$v_0$", r"$D$", r"$x_0$"],
220                                  quantiles=[0.16, 0.5, 0.84],
221                                  show_titles=True,
222                                  title_fmt=".2f",
223                                  title_kwargs={"fontsize": 12})
224 plt.show()
225
226 #####
227 # Computing RMSE over the chain for single-fault
228 #####
229
230 burned_chain_single = chain_single[burn_in_single:]
231 Ns_single = len(burned_chain_single)
232 rmse_values_single = np.zeros(Ns_single)
233
234 for i in range(Ns_single):
235     v0_i, D_i, x0_i = burned_chain_single[i]
236     model_pred = velocity_single_fault(x_data, v0_i, D_i, x0_i)
237     residuals = (y_data - model_pred) / sigma
238     rmse_values_single[i] = np.sqrt(np.mean(residuals**2))
239
240 # The "Theoretical" RMSE should be around 1.0 if the model & sigma are correct
241
242 print("Single-fault: Average RMSE = ", np.mean(rmse_values_single))
243
244 # Plot a histogram of RMSE
245 plt.figure()
246 plt.hist(rmse_values_single, bins=20, density=True)
247 plt.xlabel("RMSE (Single-Fault)")
248 plt.ylabel("Density")
249 plt.title("RMSE Distribution, Single-Fault Model")
250 plt.show()
251
252 #####
253 # Plot data + a few random posterior draws for single-fault
254 #####
255
256 x_grid = np.linspace(-100, 100, 400)
257 plt.figure()
258 plt.plot(x_data, y_data, 'o', label="Data")
259 # Plot 100 random samples from posterior
260 idxs = np.random.choice(len(chain_single_post), 100, replace=False)

```

```

261 for i in idxs:
262     v0_i, D_i, x0_i = chain_single_post[i]
263     plt.plot(x_grid, velocity_single_fault(x_grid, v0_i, D_i, x0_i), alpha
264             =0.05)
265 plt.xlabel("x")
266 plt.ylabel("velocity")
267 plt.title("Single-Fault Model: Posterior Samples")
268 plt.legend()
269 plt.show()
270 #####
271 # 6) Two-fault model
272 #####
273 # initial guess that satisfies the prior constraints:
274 # [v10, D1, x10, v20, D2, x20]
275
276 initial_guess_two = [40, 10, 0, 40, 10, 50]
277 step_scale_two = 1.0
278 n_samples_two = 40000
279
280 chain_two, log_posts_two, acc_rate_two = metropolis_sampler(
281     log_likelihood_two_fault,
282     log_prior_two_fault,
283     initial_guess_two,
284     step_scale_two,
285     n_samples_two,
286     x_data,
287     y_data
288 )
289
290 burn_in_two = 5000
291 chain_two_post = chain_two[burn_in_two:]
292 print("Two-fault acceptance rate:", acc_rate_two)
293
294 # Posterior means and standard deviations
295 mean_two = np.mean(chain_two_post, axis=0)
296 std_two = np.std(chain_two_post, axis=0)
297 print("Two-fault posterior means:", mean_two)
298 print("Two-fault posterior stds: ", std_two)
299
300 # Triangle plot (corner plot) for two-fault
301 labels_two = [r"$v_{0}^{\{1\}}$", r"$D_1$", r"$x_{0}^{\{1\}}$",
302              r"$v_{0}^{\{2\}}$", r"$D_2$", r"$x_{0}^{\{2\}}$"]
303 fig_corner_two = corner.corner(chain_two_post,
304                               labels=labels_two,
305                               quantiles=[0.16, 0.5, 0.84],
306                               show_titles=True,
307                               title_fmt=".2f",
308                               title_kwargs={"fontsize": 12})
309 plt.show()
310
311 #####
312 # Compute RMSE over the chain for two-fault
313 #####
314 Ns_two = len(chain_two_post)
315 rmse_values_two = np.zeros(Ns_two)
316
317 for i in range(Ns_two):
318     v10_i, D1_i, x10_i, v20_i, D2_i, x20_i = chain_two_post[i]
319     model_pred = velocity_two_fault(x_data, v10_i, D1_i, x10_i, v20_i, D2_i,
320                                   x20_i)
321     residuals = (y_data - model_pred) / sigma
322     rmse_values_two[i] = np.sqrt(np.mean(residuals**2))

```



```

322
323 print("Two-fault: Average RMSE = ", np.mean(rmse_values_two))
324
325 # Plot a histogram of RMSE
326 plt.figure()
327 plt.hist(rmse_values_two, bins=50, density=True)
328 plt.xlabel("RMSE (Two-Fault)")
329 plt.ylabel("Density")
330 plt.title("RMSE Distribution, Two-Fault Model")
331 plt.show()
332
333 #####
334 # Plot data + a few random posterior draws for two-fault
335 #####
336 plt.figure()
337 plt.plot(x_data, y_data, 'o', label="Data")
338 idxs_two = np.random.choice(len(chain_two_post), 100, replace=False)
339 for i in idxs_two:
340     v10_i, D1_i, x10_i, v20_i, D2_i, x20_i = chain_two_post[i]
341     plt.plot(x_grid,
342              velocity_two_fault(x_grid, v10_i, D1_i, x10_i, v20_i, D2_i, x20_i)
343              ,
344              alpha=0.05)
345 plt.xlabel("x")
346 plt.ylabel("velocity")
347 plt.title("Two-Fault Model: Posterior Samples")
348 plt.savefig('Two-Fault.png')
349 plt.legend()
350 plt.show()
351 #####
352 # Compare the models
353 #####
354 print("Single-Fault Model mean RMSE:", np.mean(rmse_values_single))
355 print("Two-Fault Model mean RMSE:", np.mean(rmse_values_two))

```

5 Data

The file `data.txt` (200 lines, comma-separated) is provided as supplementary material.